

单元二：计算与算法

第四章 计算过程与算法（一）





4.1 计算过程与算法

4.2 算法的基本组成

4.3 算法的描述方法

4.4 计算模型（迭代递推）

4.5 计算模型（穷举法）

4.6 算法的基本方法

4.7 自动机计算模型



一. 计算

- 计算机求解问题的本质是：什么可计算？
- 计算可以看作一个过程, 描述为从一个符号行 ξ 得出另一个符号行 η . ---**变换**
- “计算”就是遵循一定的规则, 用刻板、机械的动作进行的过程。 ---**基于规则的变换**
 - 例：对输入行 $12+3$ 进行计算，得出15。
 - 例：对一行汉字进行计算，得出英文。
 - 例：输入公理和规则，得出结论（定理证明）



- 例1：编程计算：求 $1 \times 2 \times \cdots \times n$ ，即 $n!$
- 分析：
 - 设置2个变量：i计数器初始值为1，p保存计算结果初始值为1；
 - 核心操作就是两数相乘 $p=i \times p$ ，反复相乘10次，，每相乘一次i的值加1。
- 根据上述分析，本题可利用循环结构求解：
第1次循环用于求 $1!$ ，第2次循环在第1次循环基础上求 $2!$ ，...,第n次循环在第n-1次循环基础上求 $n!$



例1：编程计算 $n!$ 源程序

```
#include<stdio.h>
int main()
{
    int n, i, p; //n: 存储要求阶乘的数; p: 存储求得的阶乘;
    printf("input n:\n");
    scanf("%d",&n);
    i=1;
    p=1;
    while (i <= n){
        p=i*p;
        i=i+1;
    }
    printf("%d!=%d",n,p);
    return 0;
}
```



例1 求 $1 \times 2 \times \cdots \times n$ ，即 $n!$

定义两个变量 p 和 i ， p 代表阶乘结果, i 代表本次循环要求的是 $i!$;

循环体: $p = i * p$; (求 $i!$)

$i!$

$(i-1)!$

$i = i + 1$;

循环条件: $i \leq n$

初始化: $i = 1$; $p = 1$

由于本次循环求得的 $i * p$ 的值将作为下次循环 p 的值，故本次循环将 $i * p$ 赋值给 p ，为下一次循环做准备。

二. 计算过程（算法）

对复杂问题，直接写出能解决该问题的计算机程序是困难的，为此，人们在进行程序设计时先分析问题、理清计算思路、并对计算过程进行描述。然后再转换为计算机语言实现的程序。



求 $1 \times 2 \times \cdots \times n$ 算法思路:

S1: 读入整数 n ;

S2: 使 $i=1$; $p=1$; /*变量初始化, i 是计数, p 保存阶乘结果*/

S3: 执行 $i \times p$, 乘积仍放在变量 p 中, 即 $i \times p \Rightarrow p$;

S4: 使 i 的值加1, 即 $i+1 \Rightarrow i$;

S5: 如果 $i \leq n$, 返回重新执行步骤S3以及其后的步骤S4和S5; 否则, 执行S6。

S6: 最后输出计算得到 p 的值($n!$ 的值)。

“迭代”和“循环”：在程序设计中，重复执行同样操作的过程称为“迭代”。程序中被重复执行的程序段称为“循环”。

三.算法(Algorithm)的概念

算法： 是解决问题的步骤序列（操作序列）。

描述的是
活动和过
程

1. 起床
2. 穿衣、叠被
3. 去水房洗漱
4. 回宿舍放洗漱用品
5. 骑车去食堂
6. 排队买饭
7. 吃饭
8. 交回餐具
9. 骑车去教室

描述的是操作执行后
的状态，通过状态的
转移来描述所执行的
操作。

1. 7:20前离开宿舍
2. 7:50前离开食堂
3. 8:00前进入教室

可能的活动：
起床、穿衣叠
被、洗漱等

可能的活动：
骑车到食堂、
排队买饭、吃
饭、交回餐具



4.1 计算过程与算法

- 算法与程序的区别
 - 算法：面向人对问题求解的方法的表达；
 - 程序：面向机器用计算机语言实现并运行；
- 1) 算法设计：不使用程序设计语言，而使用一种较简单明了的表达方式（例如自然语言）设计出**求解问题**的步骤序列---**算法**。
- 2) 程序编写：根据设计并描述好的算法，使用某种程序设计语言编写对应于该算法的程序。



四. 计算机算法及其特性

- ◆ 什么是计算机可执行的操作；
- ◆ 要在计算机能力集上进行算法设计；
- ◆ 算法必须具备的五个特性：
 1. 可执行性：算法中的每一个步骤都是计算机可执行的（在计算机能力集范围内）；
 2. 确定性：算法中的每一个步骤，必须是明确定义的，不得有任何歧义性；
 3. 有穷性：算法必须在执行有穷步之后结束；
 4. 有输入信息的说明：对加工对象提要求；
 5. 有输出信息的步骤：至少要输出问题答案。



- 练习1：输入若干整数，求出最大值、最小值。
- **79 32 24 25 91 51 39 42 70 62 38 67 72 66 96 81
92 16 71 42 56 12 90 47 34 16 62 64 41 72 20 59
73 36 84 44 68 47 93 57 45 14 76 72 90 79 61 44
93 89 52 69 62 87 43 19 91 88 34 42 57 76 89 90
43 34 42 67 87 47 13**



整个问题就被转变为求n-1次两个数的最大值。

每一次都是读入一个数，再和之前求得的最大数去比较。

• 练习1：输入若干个整数

• 算法思路：采用迭代计算的方法

以求最大值为例，即：

$$- \text{Max}(N_1) = N_1$$

$$- \text{Max}(N_2, N_1) = N_1 \quad \text{if } N_2 < N_1 \\ = N_2 \quad \text{if } N_2 \geq N_1$$

$$- \text{Max}(N_3, N_2, N_1) = \text{Max}(N_3, \text{Max}(N_2, N_1))$$

....

$$- \text{Max}(N_n, N_{n-1}, \dots, N_2, N_1) = \\ \text{Max}(N_n, \text{Max}(N_{n-1}, \dots, N_2, N_1))$$

定义变量max来存储前n-1个整数的最大值。第n个数将和max比较，决定前n个数的最大值。

输入若干个（假设10个）整数，求出最大值、最小值

循环条件： $i \leq 10$

循环体：

读入第*i*个整数*n*

*n*和之前最大值、最小值比较

如果 $n > \max$, 则 $\max = n$;

否则，如果 $n < \min$, 则 $\min = n$;

$i = i + 1$

循环初始化：

读入一整数*n*;

$\min = n$; $\max = n$;

$i = 2$;

请写出本题源程序



源程序：输入若干个整数，求出最大值、最小值

```
#include<stdio.h>
int main()
{
    int i, n, count, max, min; /*i:循环计数; n:读取的数; count:整数个数, max:当前最大值; min:当前最小值*/
    printf("input the count of the numbers:"); //输入整数个数
    scanf("%d",&count);
    printf("input %d numbers:\n",count);
    scanf("%d", &n);
    max = n ; //将第一个数作为最大值和最小值
    min = n ;
    i = 2; //代表接下去要读取的是第2个数
```

源程序：输入若干个整数，求出最大值、最小值

```
while ( i <= count ){  
    scanf("%d",&n); //读取第i个数  
    if (n > max) //求最大数  
        max = n;  
    else if (n < min) //求最小数  
        min=n;  
  
    i=i+1;  
}  
printf("max number is %d\n",max);  
printf("min number is %d\n",min);  
return 0;  
}
```

```
input the count of the numbers:5  
input 5 numbers:  
25 40 78 90 10  
max number is 90  
min number is 10
```

思考：为何一次性输入5个整数，也会使程序正确运行？



例2 输入120个学生的学号和成绩，要求将他们之中成绩在60分以上者的学号和成绩打印出来。

问题分析：就是反复输入处理每一个学生的信息，处理120次。

循环结束条件：

已经处理了120个学生的信息。为了计数当前输入的是第几个学生的信息，可以引入变量*i*，用于表示当前处理的学生顺序。

循环体：

- 1: 读入第*i*个学生的学号和成绩（抽象出变量num和score）；
- 2: 如果score ≥ 60 ，则打印num和score，否则不打印；
- 3: $i+1 \Rightarrow i$;



输入120个学生的学号和成绩，要求将他们之中成绩在60分以上者的学号和成绩打印出来。

S1: $1 \Rightarrow i$;

→ S2: 读入第*i*个学生的学号和成绩分别到变量num和score中;

S3: 如果score ≥ 60 , 则打印num和score ;

— S4: $i+1 \Rightarrow i$;

— S5: 如果*i* ≤ 120 , 则返回S2, 继续执行; 否则, 算法结束。

循环处理模式:

处理本次循环要做的任务;

为下一次循环做准备;



4.1 算法的概念

```
#include<stdio.h>
int main(){
    int  no, total, count ;//no:学号。total:总学生数。count: 计数
    float score;  //成绩

    printf("how many int numbers do you want to input:\n");
    scanf("%d",&total);

    count = 1;
    while( count <= total ){//进入循环体之前的count代表下一个要读取的是第几个学生信息
        printf("input no and score(no,score):\n");
        scanf("%d%f", &no, &score);
        if( score >= 60 )
            printf("no:%d\tscore:%f\n", no, score);
        count=count+1;
    }
    return 0;
}
```



4.1 算法的概念

例3 一个大于或等于3的正整数，判断它是否为一个素数（质数）。

◆所谓质数，比1大的自然数中，除了1和该数本身外，不能被其他任何自然数整除的数。例如，13是素数（质数）。因为它不能被2，3，4，...，12整除。

◆由于素数在当代的密码学中扮演了中心的作用，所以该问题具有重要意义。

◆判断一个数 n （ $n \geq 3$ ）是否为质数：将 n 作为被除数，将2到 $(n-1)$ 各个整数依次作为除数，如果都不能被整除，则 n 为素数。



看 n 能否被2到 $(n-1)$ 之间的各个整数整除：

变量抽象： n ：存放被判断的整数； i ：存放除数，取值为 $[2, n-1]$ ； r ：存放 n/i 得到的余数

循环条件：

$$i \leq n-1$$

循环初始化：

$$i=2$$

循环体

问题：当 r 为0时如何退出循环，结束算法？

n 被 i 除，得余数 r ；即 $n \bmod i = r$ ；

如果 $r=0$ ，表示 n 能被 i 整除，则打印 n “不是素数”，算法结束；否则 $i+1 \Rightarrow i$ ；



设置一个标记变量**isPrim**，初始值为1，当r为0时使**isPrim**值为0，在循环条件中加入对该变量值的判断，以决定是否提前退出循环

循环条件:

$i \leq n-1 \ \&\& \ isPrim == 1$

循环初始化:

$i=2; isPrim=1;$

循环体:

n被i除，得余数r；即 $n \bmod i \Rightarrow r$ ；

如果 $r=0$ ，表示n能被i整除，则打印n“不是素数”，令 $isPrim=0$ ；否则 $i+1 \Rightarrow i$ ；

n: 被判断的整数; **i:** 被除数;
r: 存放 n/i 得到的余数

S1: 输入 n 的值;

S2: $i=2$; /* i 是除数 */

→ S3: n 被 i 除, 得余数 r ; 即 $n \bmod i \Rightarrow r$

S4: 如果 $r=0$, 表示 n 能被 i 整除, 则打印
 n “不是素数”, 算法结束; 否则执行
 S5;

— S5: $i+1 \Rightarrow i$;

— S6: 如果 $i \leq n-1$, 返回S3; 否则, 打印 n “是
 素数”, 算法结束。

事实上, i 只需2到 $\sqrt{n}$ 之间的整数整除即可。

4.1 计算过程与算法

4.2 算法的基本组成

4.3 算法的描述方法

4.4 计算模型（迭代递推）

4.5 计算模型（穷举法）

4.6 算法的基本方法

4.7 自动机计算模型

4.3 算法的基本组成要素

- 数据对象的定义
- 表达式（算术、关系、逻辑）
- 三个基本操作（赋值、输入、输出）
- 三种控制结构(顺序、分支、循环)
- 函数调用（子程序）



问题1: 你会吃蟹黄汤包吗?

轻轻提,慢慢移,先开窗,再喝汤。

-引自南大陈道蓄讲义

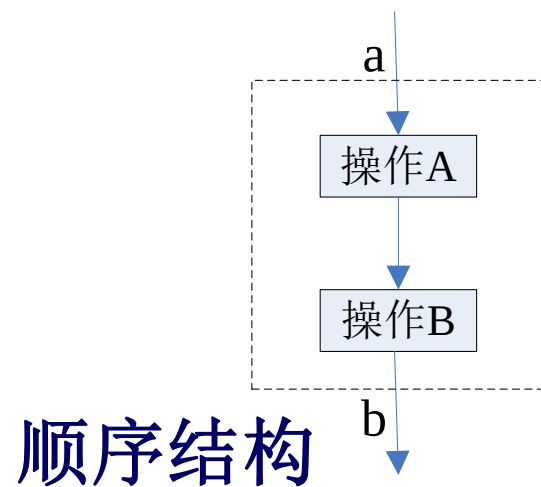


吃一只蟹黄汤包的“算法”

- 顺序很重要：---顺序结构
 1. 将包子从蒸笼中轻轻提起，and then
 2. 将包子慢慢移动到面前的小碟子中，and then
 3. 在包子的正上方咬开一个小口，and then
 4. 通过小口吸食包子里的汤(当心别烫着)，and then
 5. 将包子送入口中。完成！

-引自南大陈道蓄讲义

4.2 算法的基本组成要素

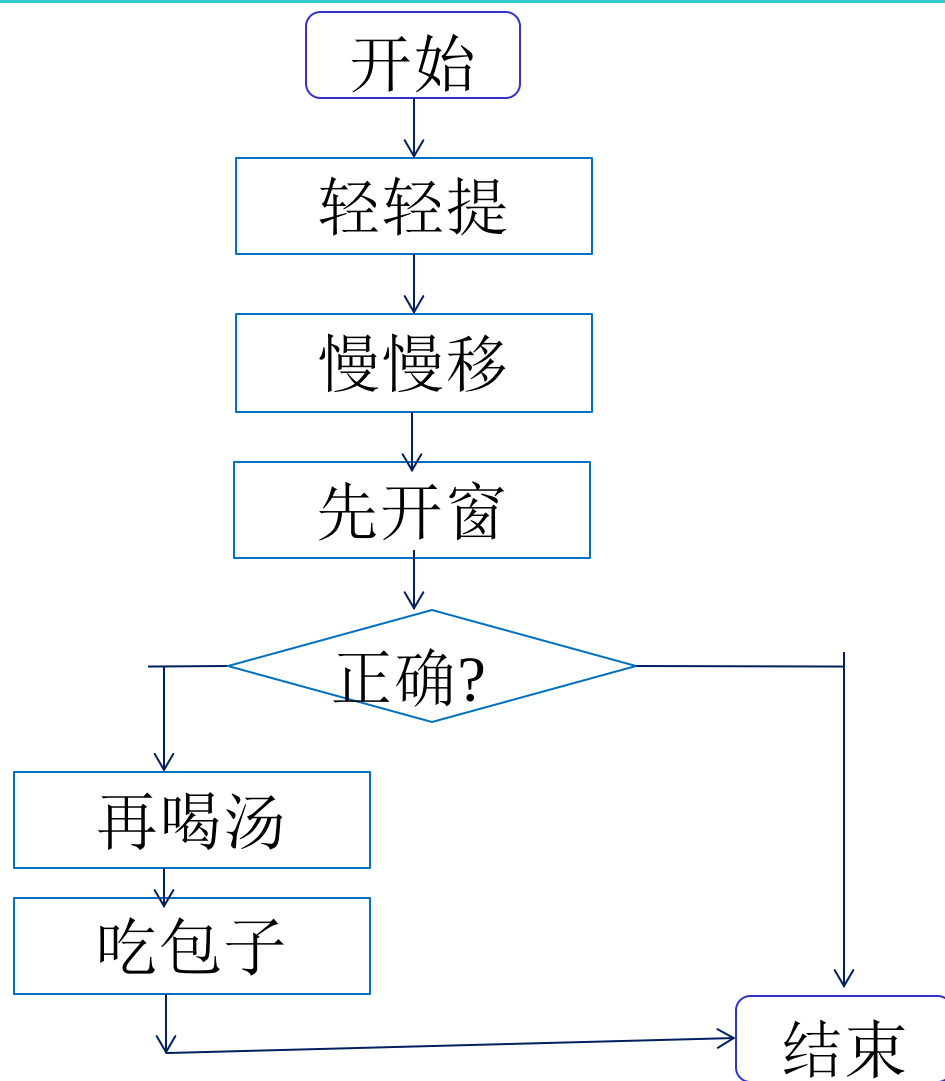


按书写顺序执行的操作构成的程序段。

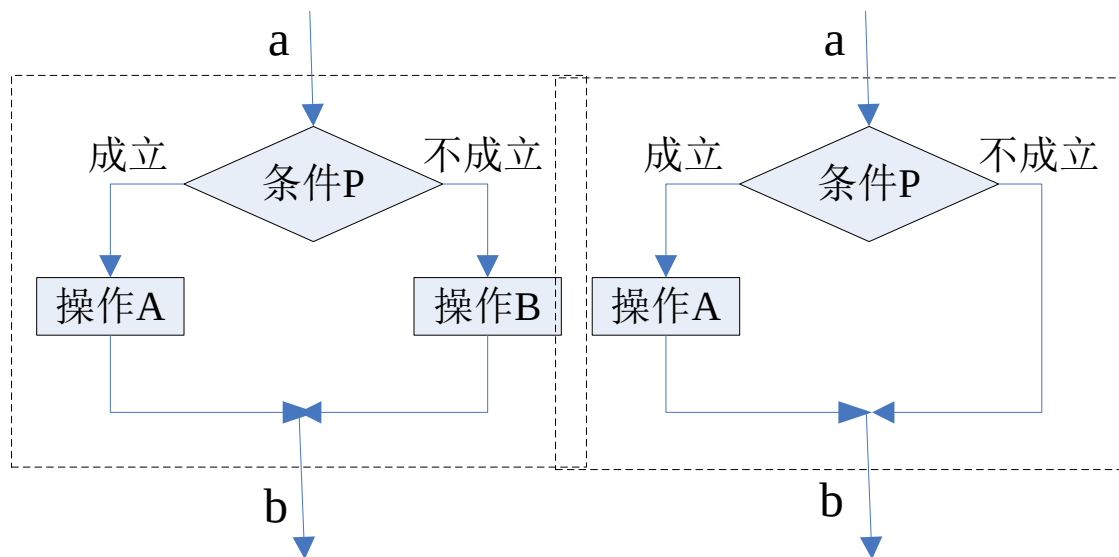


问题2: 你如何确保过程无误?

假如我们认为在步骤4和5执行前要确保前面的结果是正确的，可以在相应的地方设个“监视哨” – “guard”。



4.2 算法的基本组成要素



选择结构

◆ 根据给定的表达式是否成立而选择执行操作A或操作B。如果表达式成立，则执行操作A；如果表达式不成立，则执行操作B。

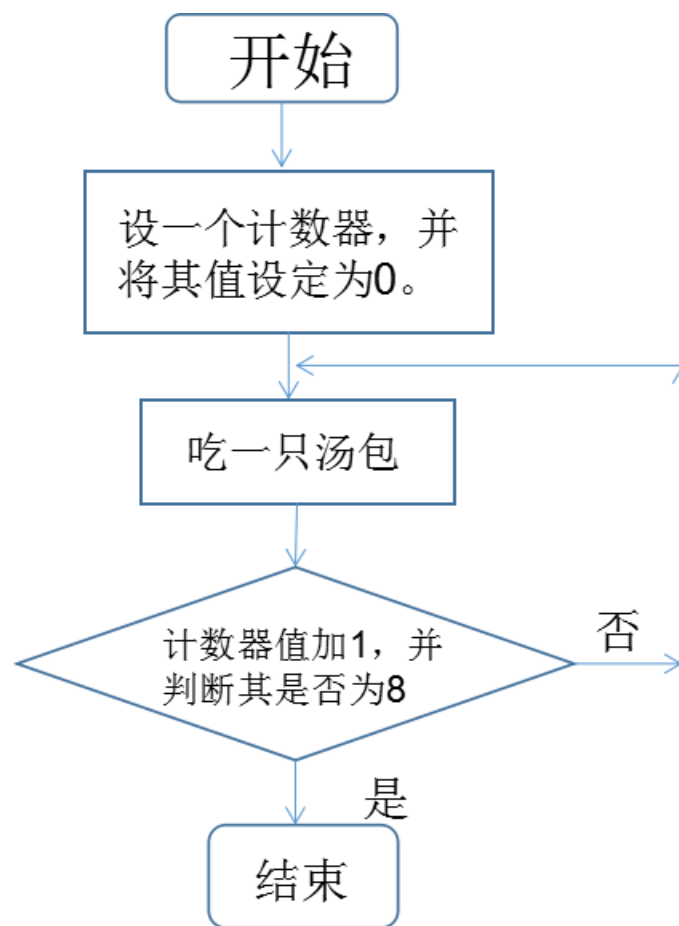
◆ 操作B可以为空。

问题3:

但是我们并不只是吃一只，
那怎么办呢？

策略一：控制数量

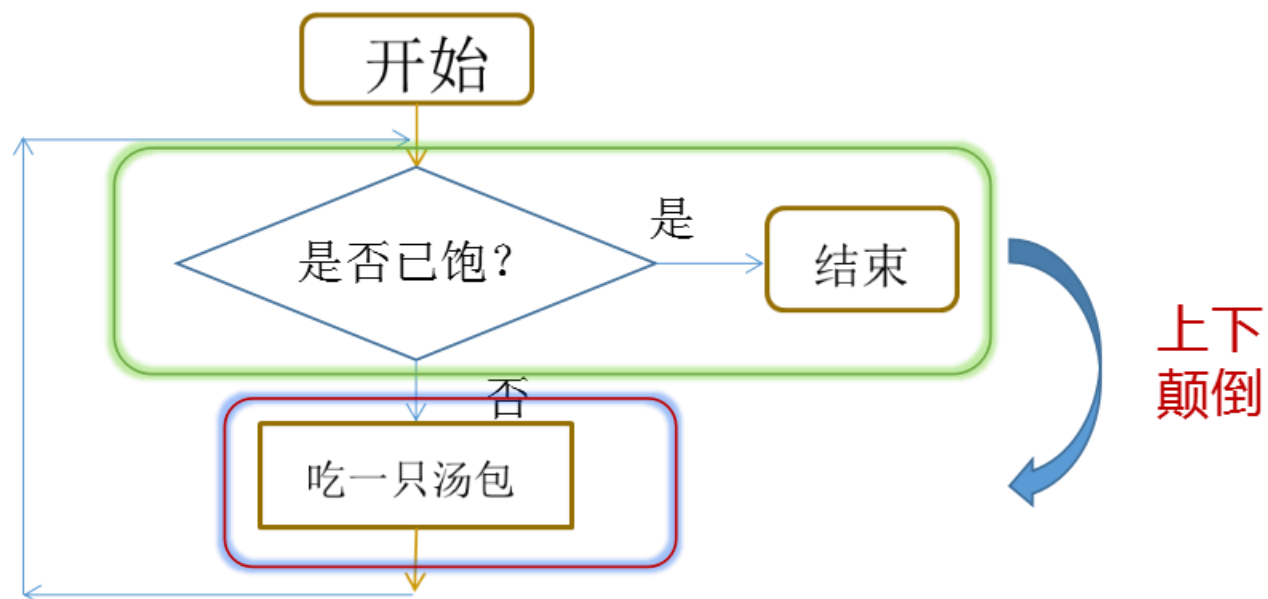
假如规定吃8只：



注意：
这个过程的“结构”与计数器的初始值没有关系！

-引自南大陈道蓄讲义

策略二：吃饱为止

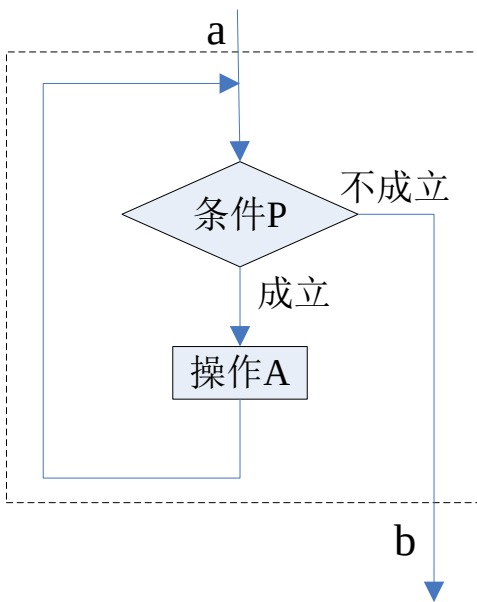


问题4:

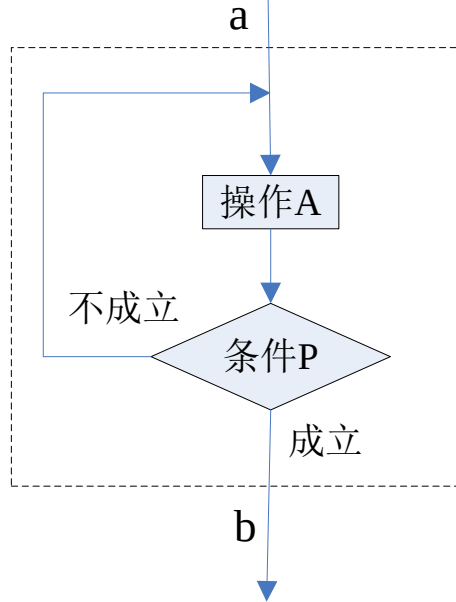
如果即使饱了，也希望至少品尝一只，该怎么办？

-引自南大陈道蓄讲义

4.2 算法的基本组成要素



当型循环结构



直到型循环结构

C语言无直到型循环结构。

比较：直到型循环结构和C语言中的do-while结构？



如何确定循环过程是正确的？

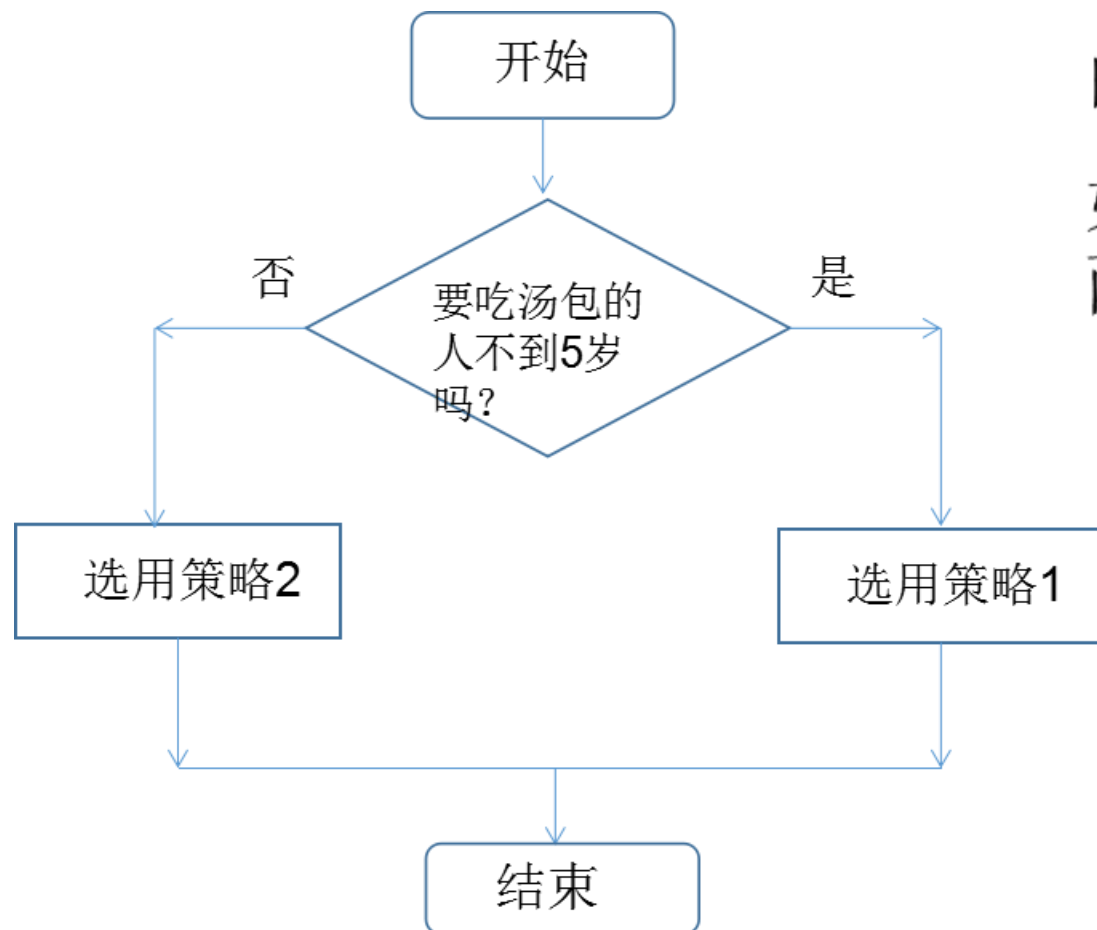
- 循环不变式
 - 这是一个逻辑表达式
 - 在循环体中它应该始终为“真”
 - 例如：用一个逐项累加的循环计算 $a*b$, 循环不变式可以是：“存放中间结果的量的值 $=a*$ 循环变量的当前值”

问题5:

你能为上述两种吃蟹黄汤包的策略选定一个循环不变式吗？

-引自南大陈道蓄讲义

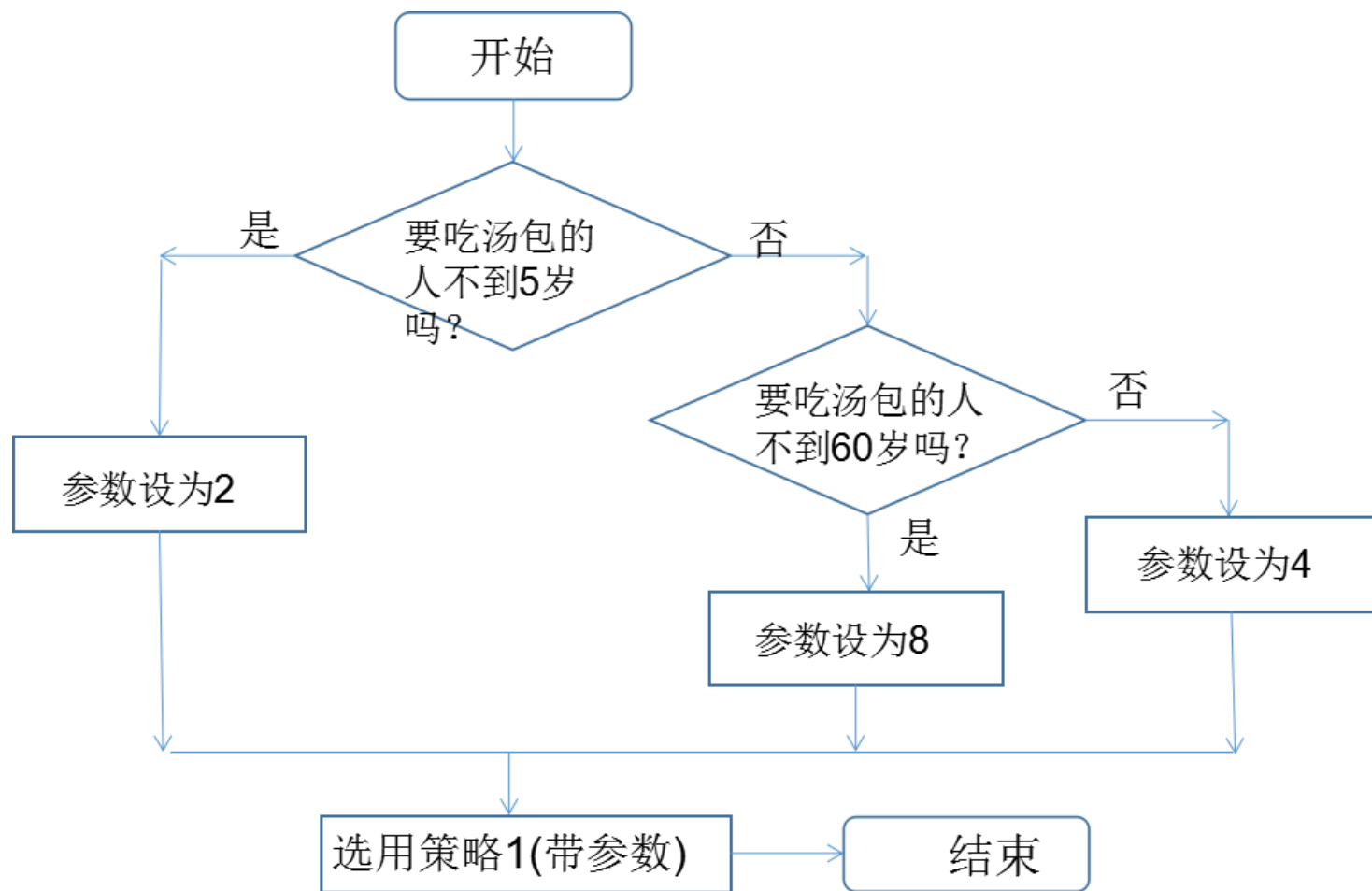
有人知道饱不饱，但有人不知道！



问题6:

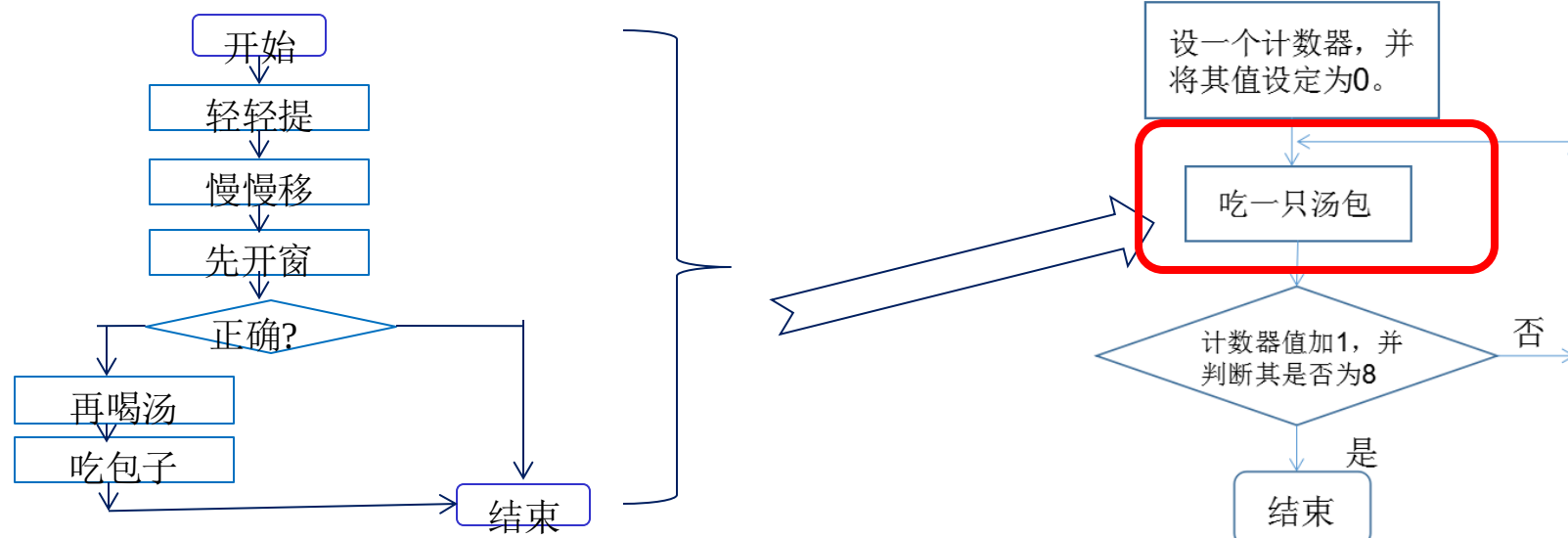
如果要判断的不止是两种可能，那怎么办？

-引自南大陈道蓄讲义



-引自南大陈道蓄讲义

问题7：可不可以将吃一只汤包的算法封装为函数，将策略选择封装为函数？



问题8：函数通常都有一种叫“形式参数”的东西，你能从“函数的参数”中想到什么？

问题9:

你能总结一下采用**subroutine**的好处吗

函数的优点：复用、封装、抽象。

丰富了算法的结构：顺序、分支、循环、调用。

- 4.1 计算过程与算法
- 4.2 算法的基本组成
- 4.3 算法的描述方法
- 4.4 计算模型（迭代递推）
- 4.5 计算模型（穷举法）
- 4.6 算法的基本方法
- 4.7 自动机计算模型



4.3 算法的描述

- 用自然语言描述
- 用流程图描述
- 用N-S流程图描述
- 用伪码描述
- 用计算机语言描述

4.3 算法的描述—自然语言

- 文字冗长;
- 不严格, 易产生歧义 (二义性);
- 不方便描述分支和循环结构;

4.3 算法的描述—流程图

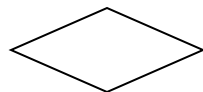
- 流程图的基本元素（ANSI规定）



起止框



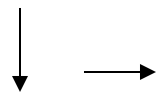
输入/出框



判断框



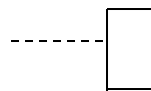
处理框



流程线



连接点



注释框

求10! 流程图

使用当型循环结构描述

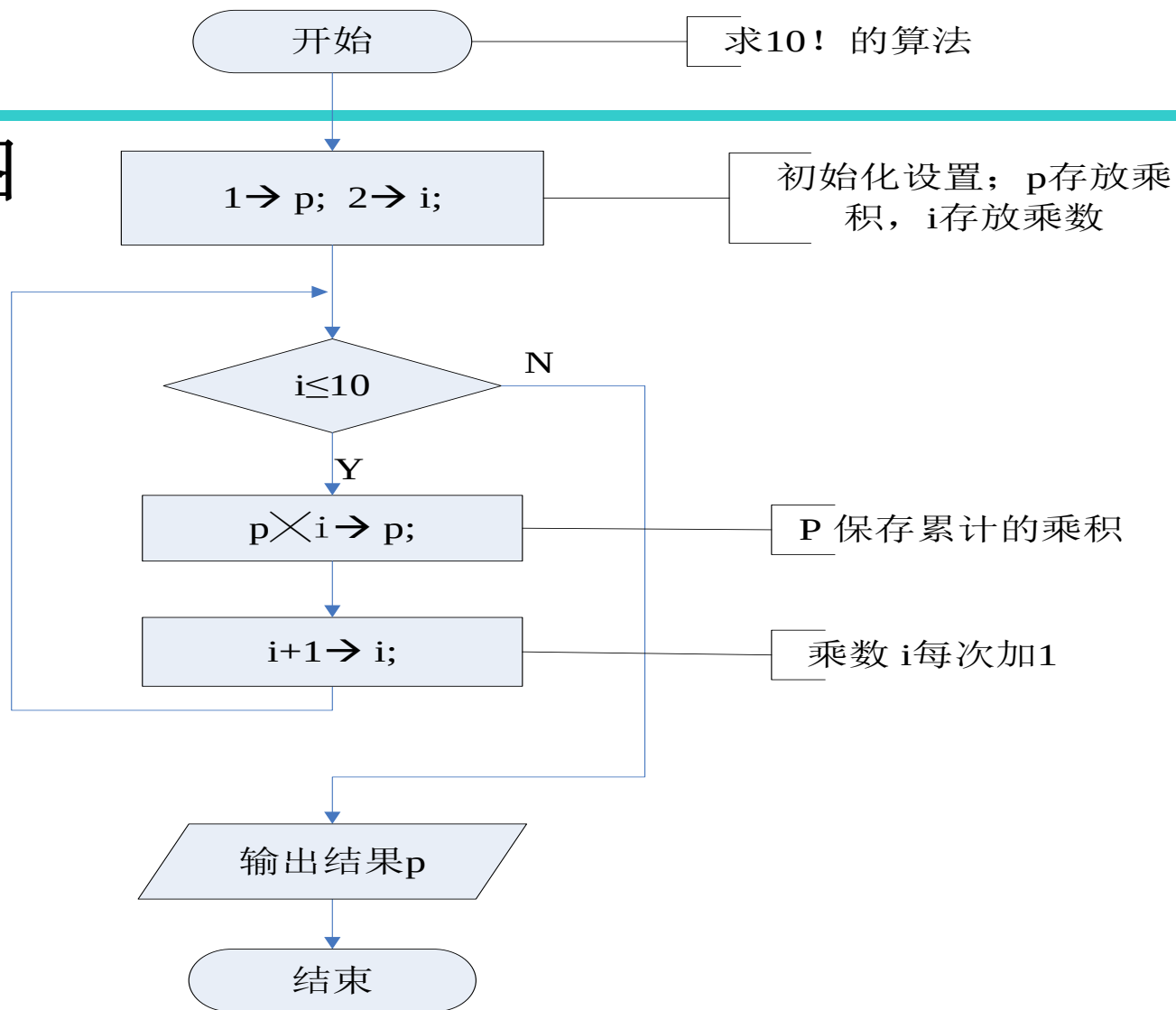


图4.10 求10! 并输出结果的算法流程图

打印学生序号 及成绩流程图

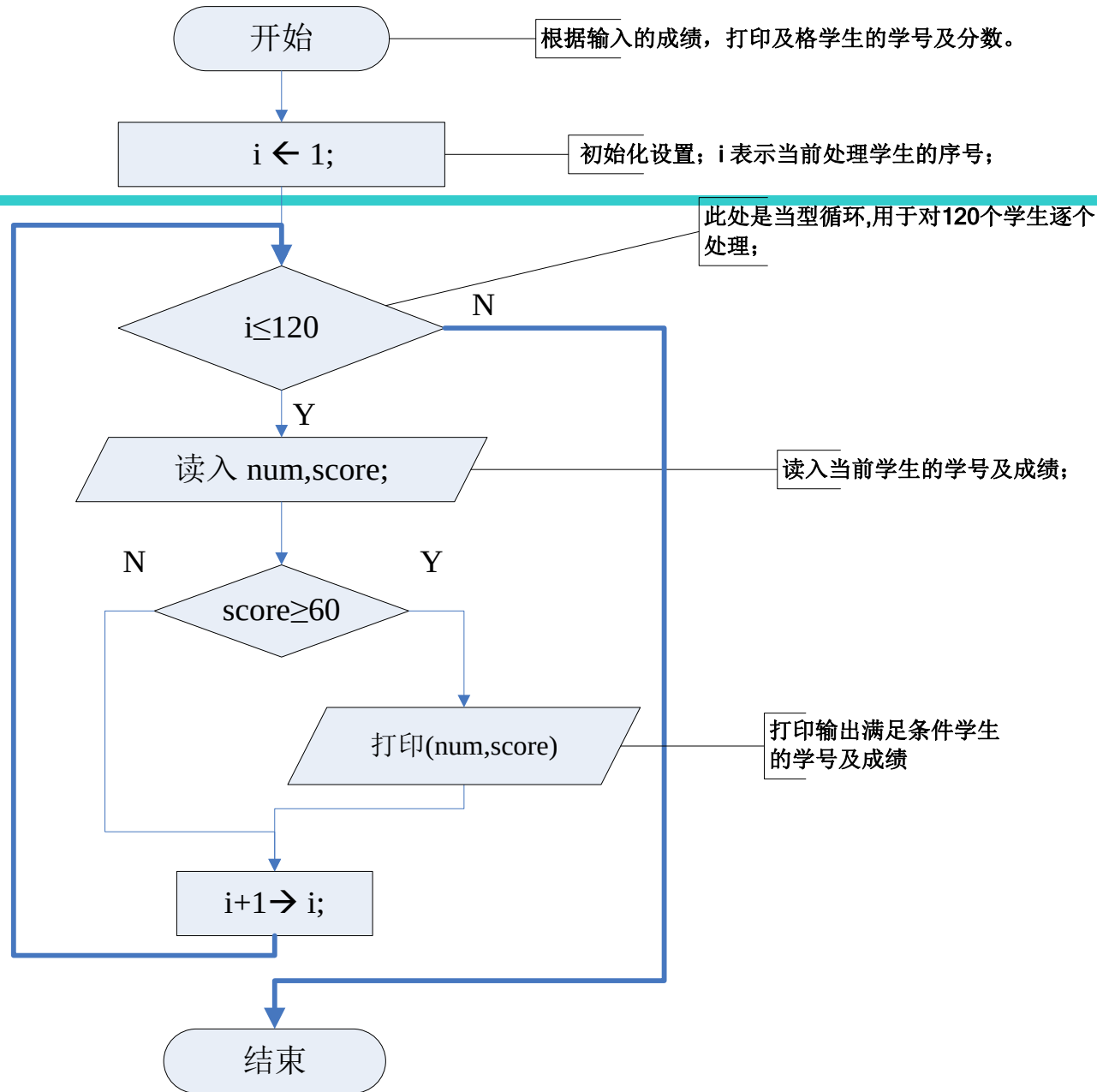


图4.11 打印及格学生的学号及成绩

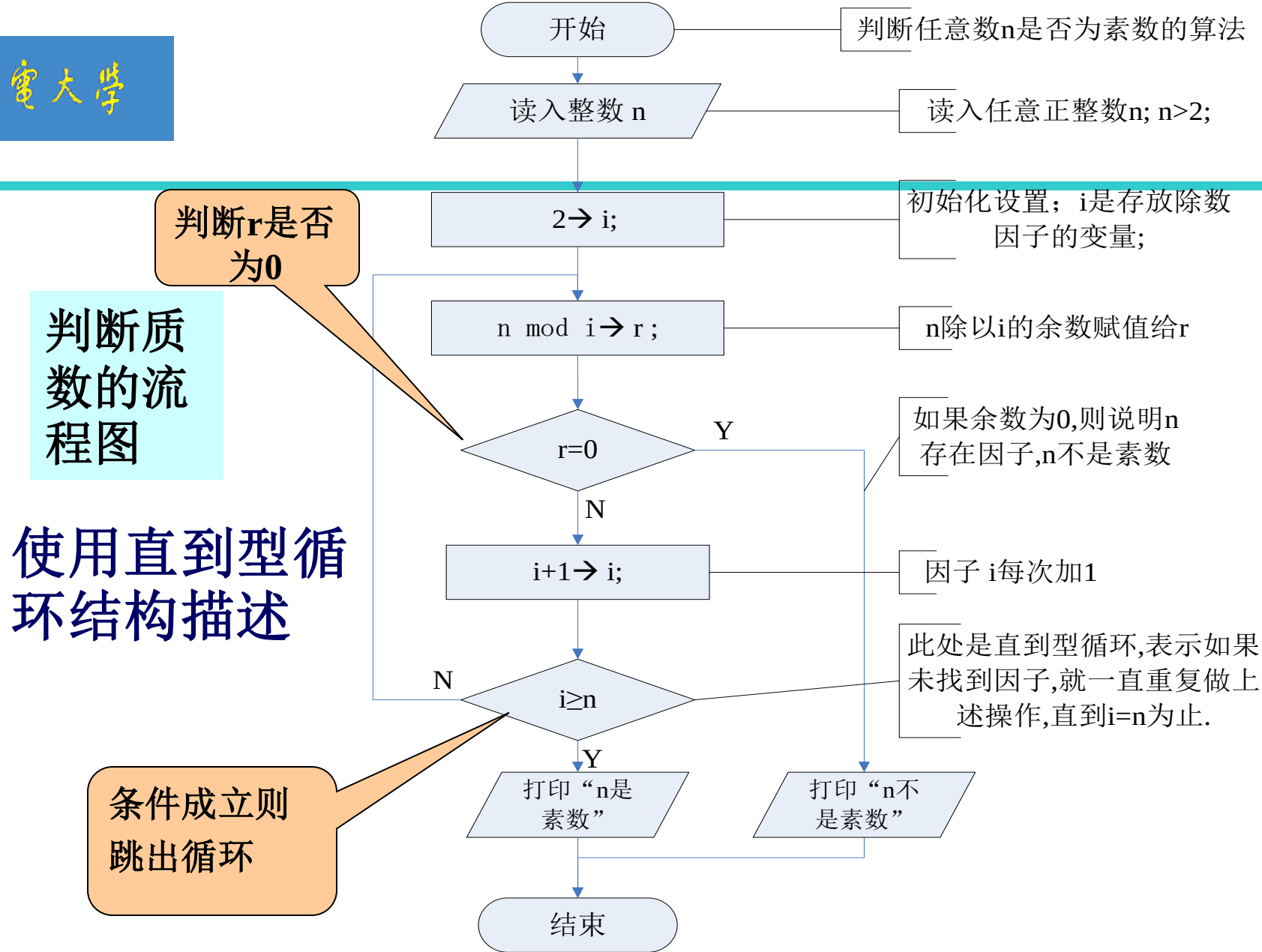
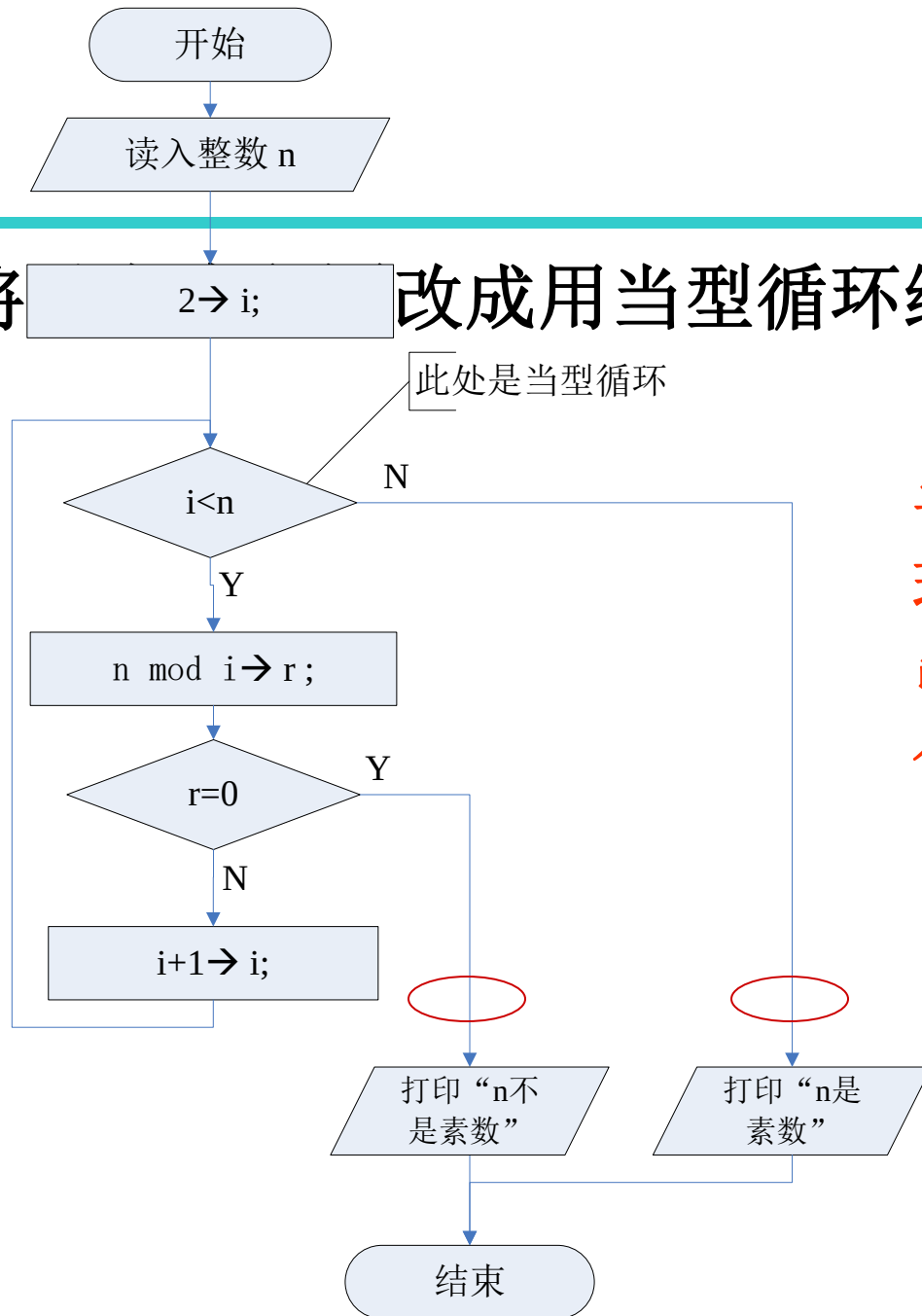


图5.14 判断任意数n是否为素数的算法流程图(非结构化)

练习：将



此处是当型循环

该流程图中循环结构有两个出口！违反单入单出原则！

如何改进？

设置标志位isPrim

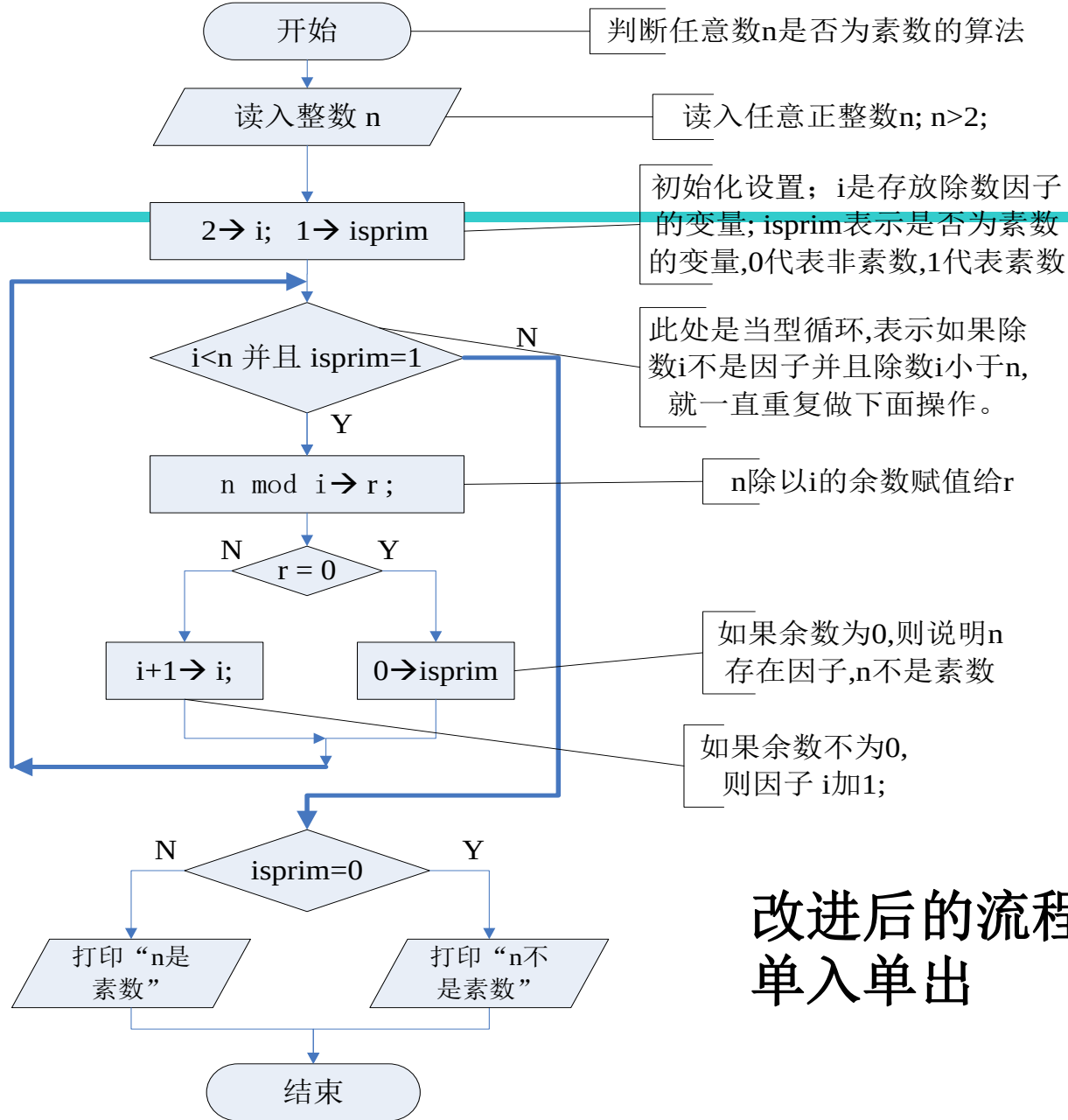
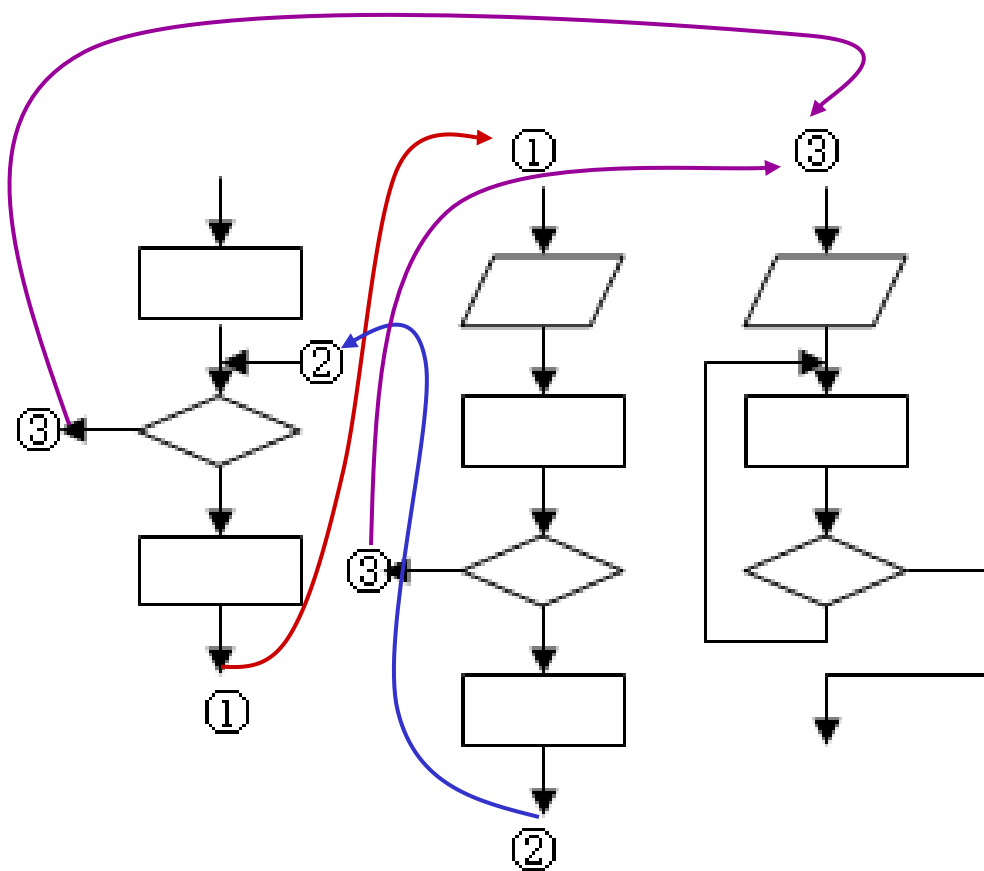


图4.21 判断任意数n是否为素数的算法流程图(结构化)



连接点（小圆圈）：用于将画在不同地方的流程线连接起来



4.3 算法的描述—N-S流程图

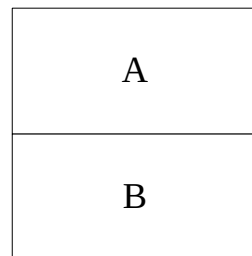
- 传统流程图的弊端
 - 对流程线的使用没有严格限制，阅读困难；
 - 不能保证算法结构的单入单出特性；
 - 占用篇幅较多；

必须限制箭头的滥用，让流程只能顺序执行下去！

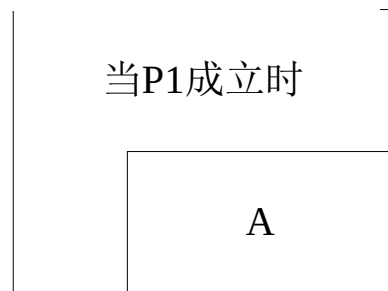
保证结构化原则的流程描述工具—N-S图

4.3 算法的描述—N-S流程图

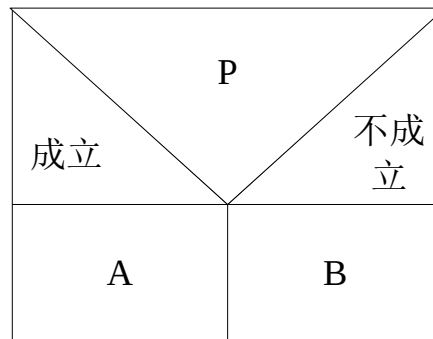
- 基本结构



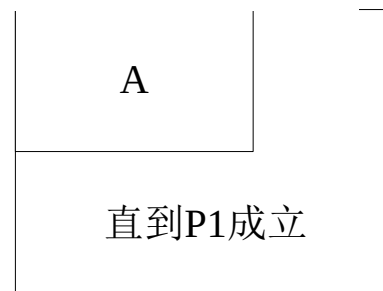
顺序结构



当型循环结构



选择结构



直到型循环结构

p、p1表示的是判断条件；**A、B**框是操作；

注意：A、B框可以是一个简单的操作（如读入数据或打印输出等），也可以是三种基本结构之一

4.3 算法的描述—N-S流程图

例1 求10!的N-S流程图

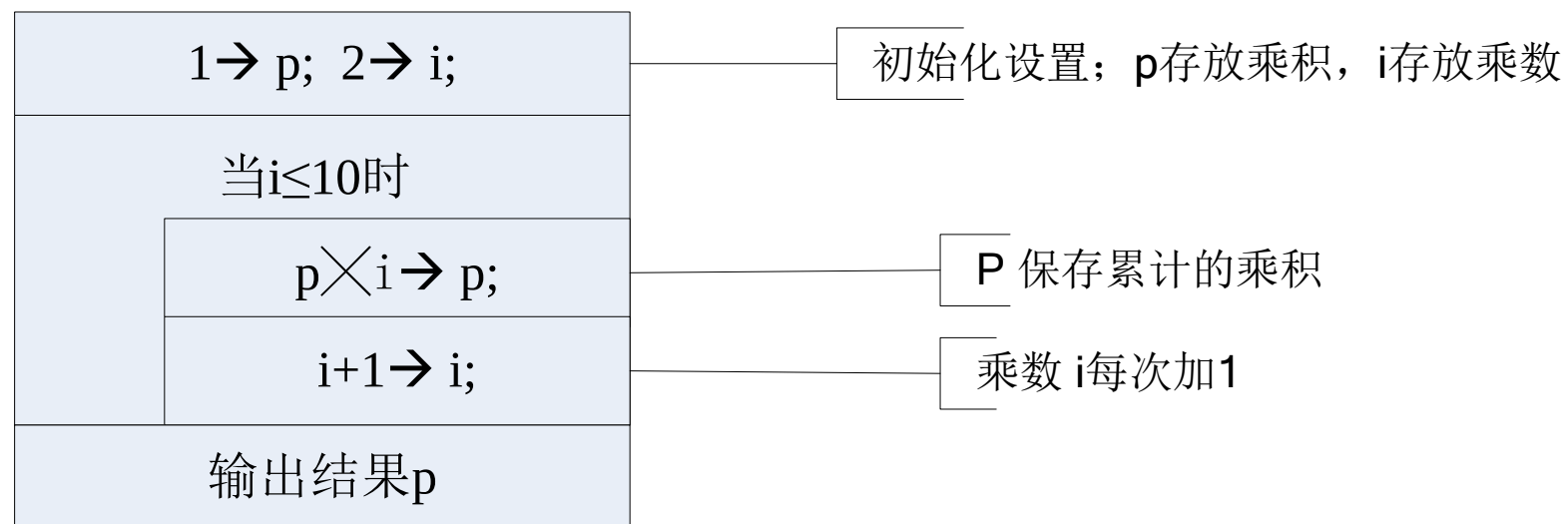


图4.19 求10! 并输出结果的算法N-S流程图

4.3 算法的描述—N-S流程图

例2 打印学生成绩N-S流程图

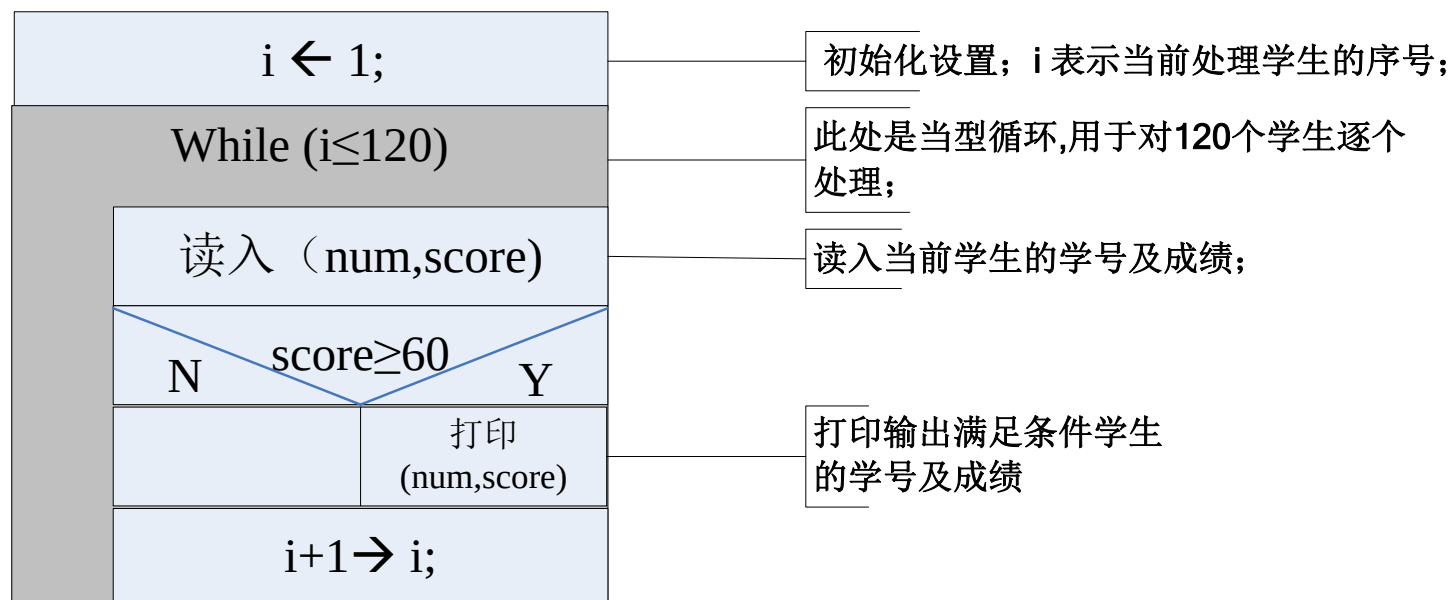


图4.20 打印及格学生的学号及成绩

4.3 算法的描述方法—N-S流程图

例3 判断质数的N-S流程图

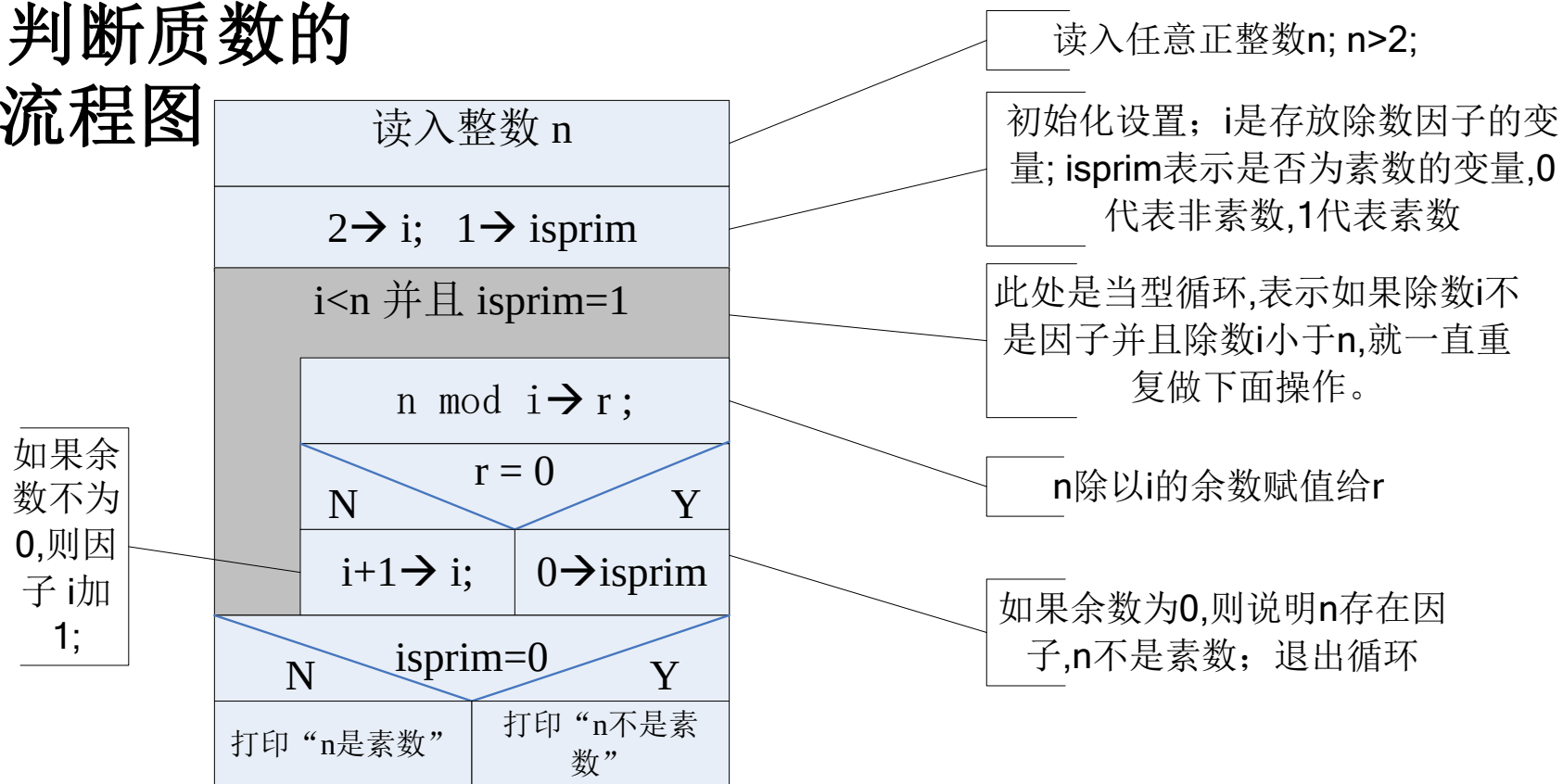


图4.23 判断任意数 n 是否为素数的算法流程图(结构化)

4.3 算法的描述—伪码描述

- 流程图和N-S图画起来比较费事，适合于表示算法，而在算法设计中使用不是很理想。
- 伪码

用介于自然语言和程序设计语言之间的文字和符号来描述算法。

```
IF x is positive THEN
    print x
ELSE
    print y
```

```
WHILE i <= 120
{   input score;
    IF score > 60 THEN
        print score
    i加1
}
```